Dokumentacja Techniczna — Planer Efektów w Bibliotece Tower-Defense dla Unity

Zespół Projektowy `TowerDefenseToolkit`

17 października 2025

Spis treści

1 EffectPlanner.cs – Dokumentacja techniczna **2**
1.1 Opis ogólny 2
1.2 Zależności 2
1.3 Struktura klasy 2
1.4 Struktura wspierajaca: EffectActionFactory 3
1.5 Interfejs: IEffectAction 4
1.6 Integracja z EffectRegistry 4

1 EffectPlanner.cs – Dokumentacja techniczna

1.1 Opis ogólny

Plik `EffectPlanner.cs` odpowiada za logike planowania, nakładania i wykonywania efektów powiazanych z `Elementami` przechowywanymi w `EffectRegistry`. Planer jest warstwa pośrednia pomiedzy systemem zdarzeń w grze (np. `CombatEvent`, `ProjectileHit`) a rejestrem efektów. Jego zadaniem jest:

- interpretacja zawartości `Elementu` (czyli mapy `<string, float> effects`),
- dynamiczne łaczenie wielu `Elementów` (stackowanie efektów),), jeżeli sa typu, który nie jest w rejestrze,
- określanie kolejności i reguł interakcji pomiedzy efektami,
- tworzenie sekwencji akcji (np. wizualnych lub mechanicznych) wynikajacych z efektów.

Planer nie przechowuje efektów bezpośrednio — pobiera je z `EffectRegistry`, a nastepnie tworzy obiekty wykonawcze (instancje klas implementujacych interfejs `IEffectAction`) do ich realizacji.

1.2 Zależności

- `EffectRegistry.cs` – źródło definicji `Elementów`,
- `EffectAction.cs` – zbiór konkretnych implementacji akcji efektów (np. `BurnAction`, `SlowdownAction`),
- `EffectStackPolicy.cs` – polityka łaczenia wielu efektów,
- `EffectContext.cs` – dane środowiskowe wywołania (np. źródło ataku, cel, pozycja).

1.3 Struktura klasy

Sekcja: Podstawowe pola i konstrukcja

- `private EffectRegistry registry`
  Wskaźnik do aktywnego rejestru efektów. Pozwala na pobieranie `Elementów` według nazwy lub pozycji.
- `private EffectStackPolicy stackPolicy`
  Obiekt definiujacy reguły łaczenia efektów (np. `Additive`, `Override`, `Multiplicative`).
- `private List<Element> activeElements`
  Lista aktualnie aktywnych elementów (pochodzacych z rejestru), które maja zostać nałożone na dane wydarzenie.
- `private List<IEffectAction> plannedActions`
  Lista akcji do wykonania, wygenerowana na podstawie `activeElements`.

```
public class EffectPlanner
{
    private readonly EffectRegistry registry;
    private readonly EffectStackPolicy stackPolicy;
    private readonly List<Element> activeElements = new();
    private readonly List<IEffectAction> plannedActions = new();

    public EffectPlanner(EffectRegistry registry, EffectStackPolicy
        policy)
    { ... }
}
```

Sekcja: Dodawanie i łaczenie elementów

- `public void RegisterElement(string elementName)`
  Pobiera `Element` z rejestru na podstawie nazwy i dodaje go do listy `activeElements`. Jeśli element o tej nazwie nie istnieje, rzucany jest wyjatek.

- `public Element CombineElements()`
  Tworzy nowy, tymczasowy `Element`, bedacy wynikiem połaczenia wszystkich `activeElements` zgodnie z aktualna `stackPolicy`. Wynikowy element nie jest zapisywany w rejestrze — istnieje tylko w kontekście bieżacego planowania.

```
public void RegisterElement(string elementName)
{ ... }

public Element CombineElements()
{ ... }
```

Sekcja: Planowanie i wykonywanie

- `public void BuildPlan(EffectContext context)`
  Tworzy liste akcji (`IEffectAction`) na podstawie połaczonego elementu. Każdy wpis w `effects` (np. `"burn": 2.0`) zostaje zmapowany na konkretna akcje — np. `new BurnAction(2.0)`.

- `public void ExecutePlan(EffectContext context)`
  Wykonuje wszystkie akcje z listy `plannedActions` w określonej kolejności (lub równolegle, jeśli to określono w konfiguracji).

```
public void BuildPlan(EffectContext context)
{ ... }

public void ExecutePlan(EffectContext context)
{ ... }
```

1.4 Struktura wspierajaca: EffectActionFactory

Klasa fabryczna, której zadaniem jest mapowanie nazw efektów z rejestru (`"burn"`, `"slowdown"`, `"brightness"`) na konkretne klasy implementujace `IEffectAction`.

```
public static class EffectActionFactory
{
    public static IEffectAction Create(string effectName, float value)
    { ... }
}
```

1.5 Interfejs: IEffectAction

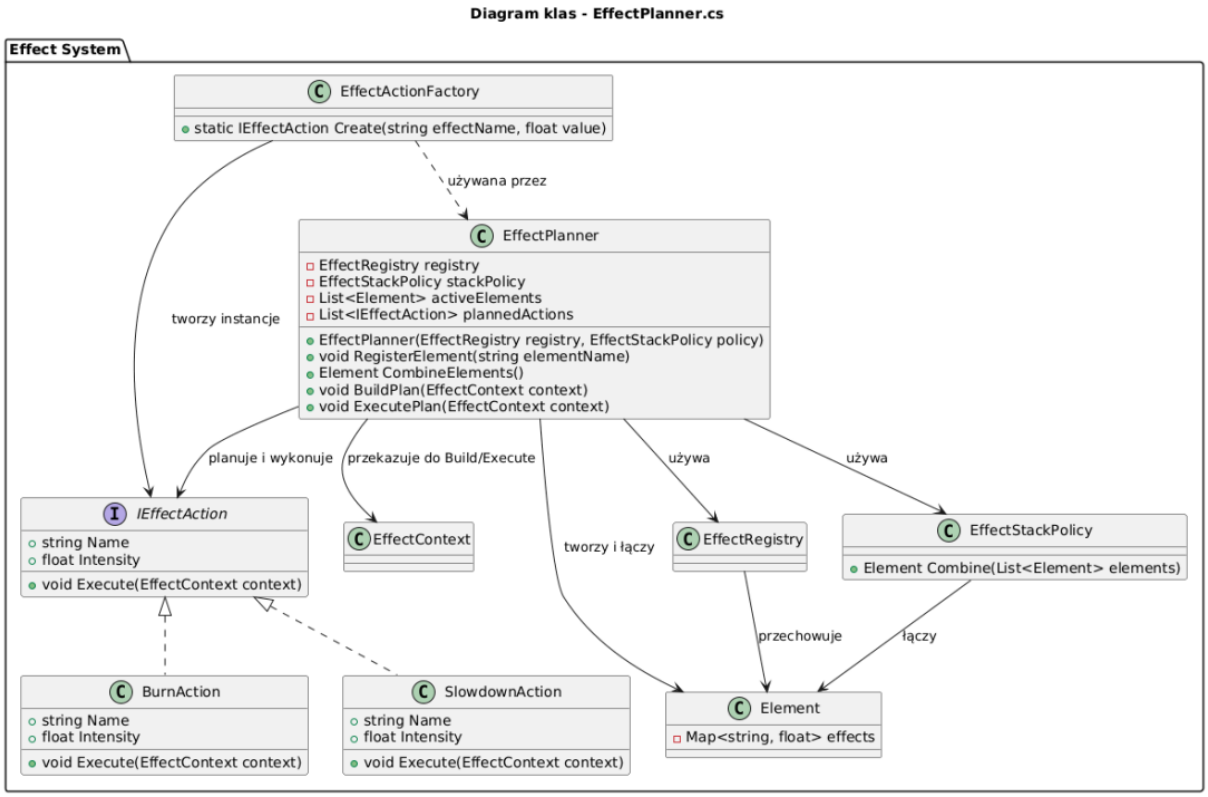
- **Opis:** Bazowy interfejs dla wszystkich efektów wykonawczych w grze.
- **Metody:**
  - `void Execute(EffectContext context)` – wykonuje akcje w kontekście bieżacego zdarzenia.
  - `string Name` – nazwa efektu.
  - `float Intensity` – siła efektu (wartość przekazana z rejestru).

```
public interface IEffectAction
{
    string Name { get; }
    float Intensity { get; }
    void Execute(EffectContext context);
}
```

1.6 Integracja z EffectRegistry

- Planer nie modyfikuje zawartości rejestru — korzysta z niego jedynie jako źródła definicji.
- Efekty wynikowe (połaczone) istnieja tylko tymczasowo i moga być generowane ponownie przy każdym wywołaniu zdarzenia.
- W przypadku checi trwałego zapisania nowego połaczenia (*Combined Element*), można użyć metody:

```
registry.putPreMadeElement(List<int> parentsId, combinedElement);
```



Rysunek 1: Diagram UML